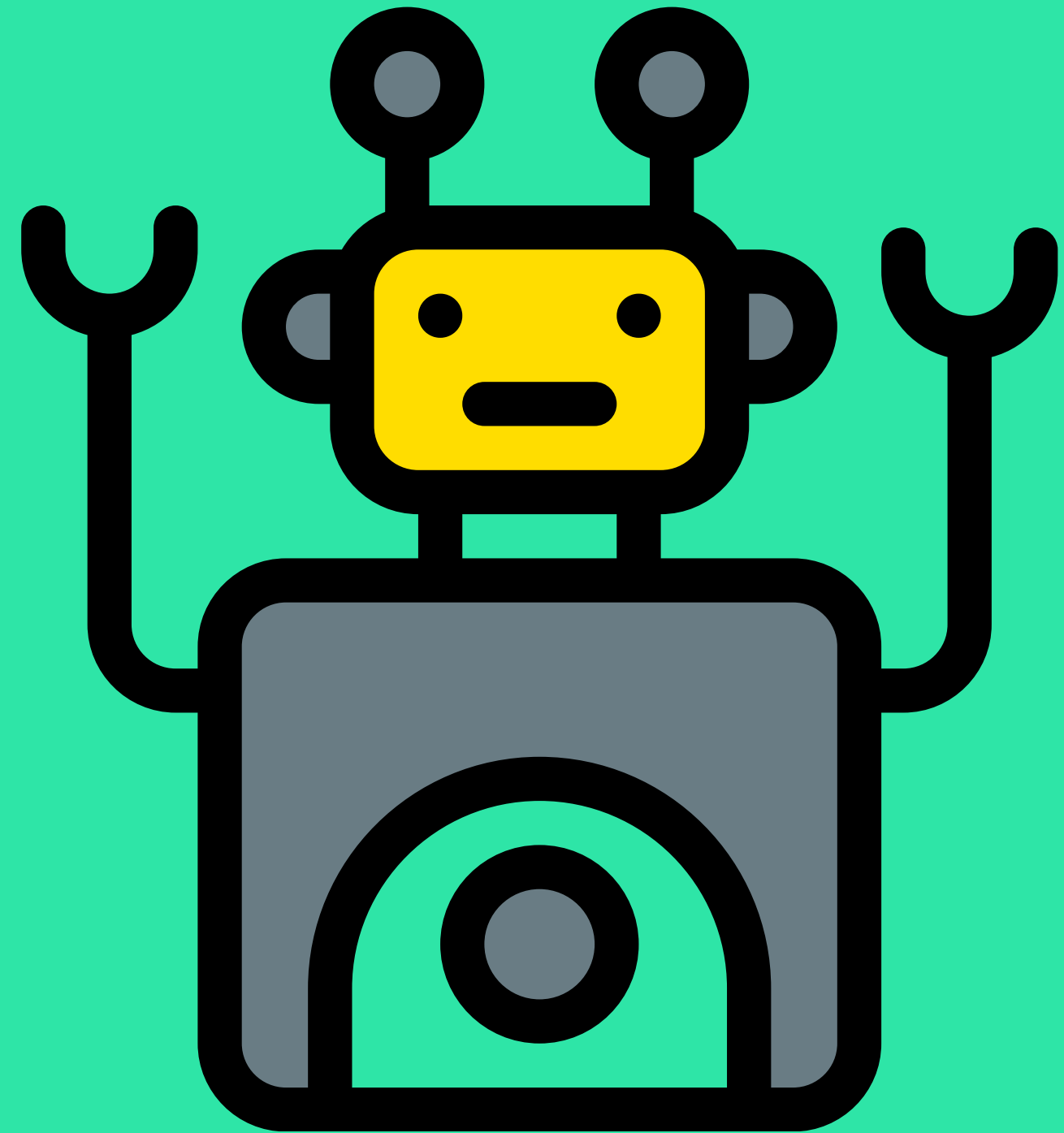


TIM ASISTEN
PRAKTIKUM

SHA-1

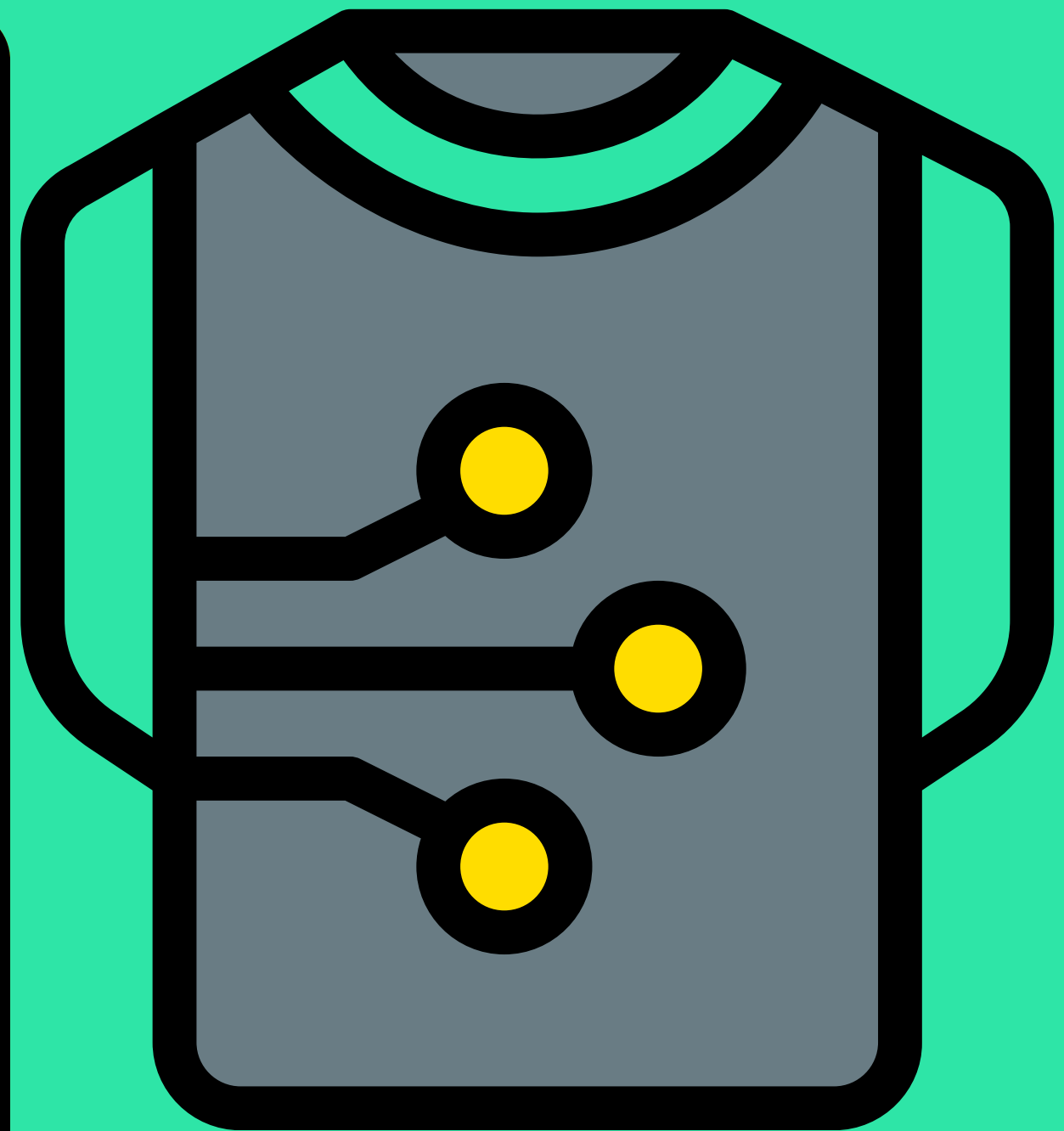
SECURE HASH ALGORITHM 1



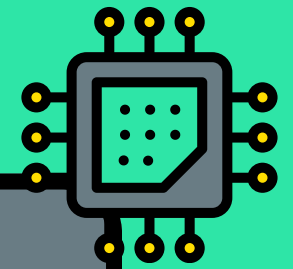
INTRODUCTION

SHA-1 (Secure Hash Algorithm 1) adalah fungsi **hash kriptografi** yang mengambil **input (pesan)** dengan **panjang berapapun** dan menghasilkan **output (hash/digest) tetap sebesar 160-bit** (setara dengan 40 karakter heksadesimal).

Algoritma ini beroperasi menggunakan operasi matematika tingkat bit (bitwise). **SHA-1 beroperasi pada kata (word) berukuran 32-bit**, dan semua operasi penjumlahan dilakukan dalam modulo **2^{32}** .



STEPS



1. PENAMBAHAN PADDING BITS

SHA-1 memproses pesan dalam blok **512-bit**. Dengan itu, kita perlu **manipulasi** bit pesan terlebih dahulu menjadi:

$$L \equiv 448 \pmod{512}$$

2. PENAMBAHAN NILAI PANJANG PESAN SEMULA

Mengapa panjang pesan wajib **448-bit**?

Karena, kita akan menambahkan sisa **64-bit** untuk menyimpan informasi **panjang asli pesan**.

3. INISIALISASI MD BUFFER

Pada pengolahan pesan, terdapat **Konstanta (K)** dan **Hash Buffer (H)** yang perlu diinisialisasikan pada putaran pertama.

4. PENGOLAHAN PESAN DALAM BLOCK BERUKURAN 512 BIT

Pengolahan dibagi lagi menjadi beberapa bagian. Pengolahan dilakukan per block (512-bit).

1. Membagi block **512-bit** menjadi 16 buah **Word** ($512/16 = 32\text{-bit}$)
2. Ekspansi **Word** dari **16 menjadi 80** buah
3. Lakukan **Siklus Utama** sebanyak **80 Putaran/Round** untuk setiap **block** nya.
4. **Update** nilai **Buffer (H)**, lalu **gabungkan** hasil **Buffer (H)** untuk mendapatkan **Hash**.

1. PENAMBAHAN PADDING BITS & 2. PENAMBAHAN NILAI PANJANG PESAN SEMULA



Input Awal (ASCII)

'a' = 0x61 = 01100001

'b' = 0x62 = 01100010

'c' = 0x63 = 01100011



Padding

1. **Tambahkan bit '1'** di akhir pesan: 01100001 01100010 01100011 **1**
2. **Tambahkan bit '0'** sampai $L \equiv 448 \pmod{512}$. Jadi, 01100011 10000000 00.....
3. **Sisa 64-bit** untuk apa? Untuk step-2, **Penambahan Nilai Panjang Pesan Pemula**
4. **Tambahkan panjang pesan asli (dalam 64-bit Big-Endian):** Panjang pesan asli kita adalah **24-bit = 0x18**. Sehingga **0x00 00 00 00 00 00 00 18**
5. **Jadi totalnya adalah 512 bit (64 bytes)**

```
// LANGKAH 1: Penambahan Padding Bits
// Tambahkan bit '1' (0x80 dalam byte)
data.push_back(0x80);
// Tambahkan bit '0' sampai panjangnya bersisa 64 bit dari kelipatan 512
while ((data.size() % 64) != 56) {
    data.push_back(0x00);
}
```

```
// LANGKAH 2: Penambahan Nilai Panjang Pesan Semula (64-bit Big Endian)
for (int i = 7; i >= 0; i--) {
    data.push_back((uint8_t)(bitLength >> (i * 8)));
}
```

3. INISIALISASI MD BUFFER

```
// LANGKAH 3: Inisialisasi MD Buffer
SHA1_State state;
state.H[0] = 0x67452301;
state.H[1] = 0xEFCDAB89;
state.H[2] = 0x98BADCFE;
state.H[3] = 0x10325476;
state.H[4] = 0xC3D2E1F0;
```

```
for (int i = 0; i < 80; i++) {
    uint32_t f, k;
    if (i < 20) { f = (b & c) | ((~b) & d); k = 0x5A827999; }
    else if (i < 40) { f = b ^ c ^ d; k = 0x6ED9EBA1; }
    else if (i < 60) { f = (b & c) | (b & d) | (c & d); k = 0x8F1BBCDC; }
    else { f = b ^ c ^ d; k = 0xCA62C1D6; }
```

Inisialisasi Hash Buffer (H)

- H0=0x67452301
- H1=0xEFCDAB89
- H2=0x98BADCFE
- H3=0x10325476
- H4=0xC3D2E1F0



Inisialisasi Konstanta (K)

- Putaran 0 hingga 19:
K=0x5A827999
- Putaran 20 hingga 39:
K=0x6ED9EBA1
- Putaran 40 hingga 59:
K=0x8F1BBCDC
- Putaran 60 hingga 79:
K=0xCA62C1D6



4. PENGOLAHAN: MEMBAGI BLOCK 512-BIT MENJADI 16 BUAH WORD ($512/16 = 32\text{-BIT}$)



Hasil Padding

01100001 01100010
01100011 10000000
00000000 000.....



Membagi 512-bit block menjadi 16 buah “Word”

1. Pecah 512-bit block, menjadi 32-bit - 32-bit

2. Contoh:

a. **W0** = 0x61626380

b. **W1** = 0x00000000

c. **W2** = 0x00000000

d. ... (semuanya nol) ...

e. **W14** = 0x00000000

f. **W15** = 0x00000018 (Ini panjang pesan aslinya)

```
for (int i = 0; i < 16; i++) {  
    w[i] = (blok[i*4] << 24) | (blok[i*4+1] << 16) | (blok[i*4+2] << 8) |  
    (blok[i*4+3]);  
}
```

4. PENGOLAHAN: EKSPANSI WORD DARI 16 MENJADI 80 BUAH

Hash Buffer

W0 - W15, sedangkan kita

butuh 80 Putaran pada prosesnya. (W16 - W79)



Message Schedule

$$W_t = (W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16}) \lll 1$$

1. Misal $t = 16$

a. **W13 xor W8 xor W2 xor W0**

b. Karena $W_{2,8,13} = 0$, maka $W_0 \text{ xor } W_{..} = W_0$

c. Bentuk biner: **0110 0001 0110 0010 0110 0011 1000 0000**. Lakukan **Rotasi ke kiri sebesar 1 bit**

d. **Hasil** :1100 0010 1100 0101 1100 0111 0000 0000

e. **W16= 0xC2C4C700**

```
for (int i = 16; i < 80; i++) {  
    w[i] = ROTL(1, w[i-3] ^ w[i-8] ^ w[i-14] ^ w[i-16]);  
}
```

4. PENGOLAHAN: INISIALISASI REGISTER

Hasil Pembagian 32-bit

Kita sudah memiliki Hash

- Buffer (**H0-H4**). Tetapi, ini
- dikhususkan untuk nilai akhir
- Hash.



Register (A-E)

1. Kita masukkan nilai H0-H4 ke variabel sementara **A-E**

- a. **A=H0**
- b. **B=H1**
- c. **C=H2**
- d. **D=H3**
- e. **E=H4**

2. Variabel ini akan terjadi perubahan setiap Putarannya.

```
uint32_t a = state.H[0], b = state.H[1], c = state.H[2], d = state.H[3], e =  
state.H[4];
```


4. PENGOLAHAN: SIKLUS UTAMA 80 PUTARAN

Bahan Siap

Semua Variabel sudah diinisialisasikan, saatnya melakukan Pengolahan Pesan!



```
for (int i = 0; i < 80; i++) {  
    uint32_t f, k;  
    if (i < 20) { f = (b & c) | ((~b) & d); k = 0x5A827999; }  
    else if (i < 40) { f = b ^ c ^ d; k = 0x6ED9EBA1; }  
    else if (i < 60) { f = (b & c) | (b & d) | (c & d); k = 0x8F1BBCDC; }  
    else { f = b ^ c ^ d; k = 0xCA62C1D6; }  
  
    uint32_t temp = ROTL(5, a) + f + e + k + w[i];  
  
    e = d; d = c; c = ROTL(30, b); b = a; a = temp;  
}
```

1. Siklus Utama terdiri dari 80 Putaran untuk **1 block** (512-bit)

2. Tujuan dari per-putaran adalah menemukan: **TEMP**: $TEMP = (A \lll 5) + f + E + W_0 + K_0 \pmod{2^{32}}$

3. Rumus $f(t, B, C, d)$:

a. $0 \leq t \leq 19$: $f = (B \wedge C) \vee (\neg B \wedge D) \rightarrow$ Untuk putaran 0-19

b. $20 \leq t \leq 39$: $f = B \oplus C \oplus D \rightarrow$ Untuk putaran 20-39

c. $40 \leq t \leq 59$: $f = (B \wedge C) \vee (B \wedge D) \vee (C \wedge D) \rightarrow$ Untuk putaran 40-59

d. $60 \leq t \leq 79$: $f = B \oplus C \oplus D \rightarrow$ Untuk putaran 60-79

4. Lakukan Update variabel **Register** untuk putaran selanjutnya:

a. $E = D$

b. $D = C$

c. $C = B \lll 30$ (Nilai B diputar ke kiri 30 bit)

d. $B = A$

e. $A = TEMP$

```
e = d; d = c; c = ROTL(30, b); b = a; a = temp;
```

- $\wedge$: **AND** (Logika Dan)
- $\vee$: **OR** (Logika Atau)
- $\oplus$: **XOR** (Logika Eksklusif Atau)
- $\neg$: **NOT** (Komplemen/Kebalikan bit)
- $+$: Penjumlahan modulo 2^{32}
- $X \lll n$: **Left Rotate** (Geser putar ke kiri sebanyak n bit)

4. PENGOLAHAN: UPDATE NILAI HASH BUFFER

Sudah Selesai 80 Siklus

Proses sudah menyelesaikan
80 Putaran, dan variabel

Register siap untuk
mengupdate Hash Buffer!



1. Setelah 80 putaran selesai untuk **satu blok 512-bit** tersebut, **tambahkan** nilai **A,B,C,D,E** yang baru didapat **ke nilai hash H** kita sebelumnya:

a. $H0 = H0 + A$

b. $H1 = H1 + B$

c. $H2 = H2 + C$

d. $H3 = H3 + D$

e. $H4 = H4 + E$

2. **Jika** masih ada blok lain (misal panjang pesan asli > 448 bit), **Maka gunakan Hash Buffer** ini untuk putaran pertama blok selanjutnya.

```
state.H[0] += a; state.H[1] += b; state.H[2] += c; state.H[3] += d; state.H[4] += e;
```

5. DAPATKAN HASH NYA!



Seluruh Blok Sudah diproses

Semua blok sudah diproses, maka pengolahan pesan sudah selesai. Hash sudah bias didapatkan!



1. **Jika seluruh block** sudah diproses, maka Hash Buffer sudah berada di versi finalnya.
2. Step terakhir adalah untuk **menyatukan seluruh Buffer** tersebut (**H0 - H4**), yaitu **menggabungkan semuanya** secara **berjejer** menjadi sebuah **string panjang**
3. Hasilnya adalah **160-bit atau 40 karakter hexadecimal!**

```
// Konversi hasil ke format Hex
char hexOutput[41];
for (int i = 0; i < 5; i++) {
    sprintf(hexOutput + (i * 8), "%08x", state.H[i]);
}
return string(hexOutput);
```

TIM ASISTEN
PRAKTIKUM

THANK YOU

